UITS

Future will be better than thy past

An initiative of PHP Family

University of Information Technology & Sciences

Computer Graphics & Multimedia Lab

Project Proposal

Course Title : Computer Graphics & Multimedia Lab

Course Code : CSE0611328

Submitted to:

Teacher's Name	Designation
Engineer Robiul Islam	Assistant Professor
Tasneen Tanvir	Lecturer

Submitted by:

Student's Name	ID
Gaus Saraf Murady	04324100051010 88
Lima Akter	04324100051010 63
Fahima Akter	04324100051010 93

Semester : Autumn 2026

Batch : 55

Department : CSE

Date of Submission: 09.09.2026

Title:

The Wenderer: A Multi-Directional Endless Runner Using Modern OpenGL

Project Description:

"The Wenderer" is an interactive 2D/pseudo-3D endless runner developed in C++ using core modern OpenGL graphics principles. While traditional endless runners lock movement to a single forward vector, this project centers on dynamic disorientation, testing reflexes as the world violently shifts its traversal axis.

The visual presentation leverages basic modern OpenGL techniques: custom GLSL shader pipelines, dynamic 2D sprite-sheet rendering, and texture blending mapped onto hardware-buffered quads. To enhance player immersion and aesthetic polish, the project integrates dynamic camera shake, ambient particle bursts, and event-driven Sound Effects (SFX) triggered during turns, near-misses, and impact states using lightweight multimedia audio hooks.

Discussion:

We want to work on a game that would be an endless runner, and besides the typical obstacles and difficulties the direction of the running will keep changing as the game advances. We chose the name “The Wenderer” for the game. While a traditional wanderer travels aimlessly across a vast expanse, a *Wenderer* is defined by the deliberate, sudden, and constant **redirection** of their movement.

As distance increases, the game breaks standard endless runner tropes through the following mechanics:

1. **Dynamic Traversal Redirection (Wending Mechanism):** At procedural intervals or marker triggers, the running axis shifts instantaneously (e.g., from horizontal scrolling to vertical descents, or sharp 90-degree orthogonal pivot points). The player must instantly adjust their control mapping and depth perception relative to the new running vector.

2. **Rotational Camera Follow:** Whenever a direction switch occurs, the view matrix interpolates smoothly via rotational transforms, swinging the projection plane around the player sprite to orient the new path ahead.

3. **Vector-Adaptive Obstacle Patterns:** Barriers, collapsing tiles, and projectile hazards spawn relative to the current direction vector, requiring the collision engine to recalculate dynamic bounding boxes across varying axes.

4. **Pacing & Reaction Thresholds:** The frequency of axis transitions ramps up alongside base run velocity, making the gameplay a test of spatial orientation under cognitive load.

Technical OpenGL Implementation Details:

1. **Sprite Animation & Texture Atlasing:** Utilizing stb_image to map sprite atlases onto vertex arrays. Frame-by-frame animation is achieved via uniform-based UV coordinate offsets inside the vertex shader, driving running, turning, and collision animations.

2. **Dynamic Transformation Pipeline:** Implementing Model-View-Projection (MVP) matrices using the GLM library. Rotational matrices dynamically rotate the camera frame to follow trajectory shifts smoothly without requiring complex multi-scene reloading.

3. **Axis-Aligned Bounding Box (AABB) Collision:** Lightweight hit-box math calculated per frame in C++ to detect player contact with hazards, pickups, and redirection anchors.

4. **Audio & Feedback Systems:** Integrating a minimalist C/C++ audio library (such as miniaudio or SDL2_mixer) synchronized to OpenGL render-loop events to deliver responsive auditory cues for turns, near-misses, and game-over states.